

G0O02a: Statistical Data Analysis: Project 2 - 2025/2026 Spring (Code)

Luisa Scaroni

```
library(car)
library(moments)
library(robustHD)
library(robustbase)
library(mrfDepth)
library(readxl)
library(psych)
library(corrplot)
library(plot3D)
library(MASS)
library(Hmisc)
library(rrcov)
library(factoextra)
library(dplyr)
library(cluster)
library(fossil)
library(pheatmap)
```

Loading Data and Setup

```
set.seed(1085999)
sweetsdata <- read.csv("sweets_data.csv", stringsAsFactors = TRUE)

# Select two product categories and sample 150 observations from each
sample1 <- sample(which(sweetsdata$labels == "biscuits"), 150)
sample2 <- sample(which(sweetsdata$labels == "cakes"), 150)
mydatafull <- sweetsdata[c(sample1, sample2), ]

# Separate features and labels for clustering
X <- subset(mydatafull, select = -labels)
y_labels <- factor(mydatafull$labels)
```

Part 1: Cluster Analysis

In this part, we examine whether standard nutritional values can naturally group the snacks. The goal is to cluster the nutritional variables (excluding labels). The labels may only be used afterwards to evaluate how well the clustering matches the product categories. *## Question 1.1* We start by loading the data and

selecting the numerical variables. Then, if necessary, we scale the data before performing cluster analysis. We consider X and then study the scale of its variables. If they are too different, this means that we have to standardize/scale them:

```
summary(X)
```

```
##   energy_kcal      water  carbohydrates_available  sugars
##   Min.   :165.0   Min.    : 0.50   Min.    :14.40      Min.    :11.00
##   1st Qu.:385.0   1st Qu.: 4.00   1st Qu.:47.92      1st Qu.:25.00
##   Median :445.0   Median :13.75   Median :56.00      Median :31.00
##   Mean   :430.1   Mean    :16.69   Mean    :54.51      Mean    :31.03
##   3rd Qu.:491.2   3rd Qu.:23.00   3rd Qu.:61.00      3rd Qu.:36.00
##   Max.   :640.0   Max.    :69.30   Max.    :89.00      Max.    :63.00
##   dietary_fibres  fat_total  fatty_acids_saturated  Na
##   Min.    : 0.000   Min.    : 0.5   Min.    : 0.000   Min.    :0.0000
##   1st Qu.: 1.500   1st Qu.:14.0   1st Qu.: 5.000   1st Qu.:0.0800
##   Median : 2.000   Median :20.0   Median : 8.000   Median :0.1100
##   Mean    : 2.546   Mean     :20.0   Mean    : 8.635   Mean    :0.1522
##   3rd Qu.: 3.075   3rd Qu.:25.0   3rd Qu.:11.000   3rd Qu.:0.2000
##   Max.    :14.000   Max.     :51.0   Max.    :27.000   Max.    :0.6900
```

Now that we know that we have to scale the data, we can proceed as follows:

```
standx=scale(X)
```

Discussion:

From the summary of X (the dataset without labels) we can see that the variables are on completely different scales. For example, “Na” has a median of 0.1100, while “energy_kcal” has a median of 445. Not scaling them would imply disregarding the variables with smaller values, as their effect would be insignificant compared to the larger ones.

Question 1.2

We now calculate the dissimilarity matrix for the Euclidean and Manhattan distance:

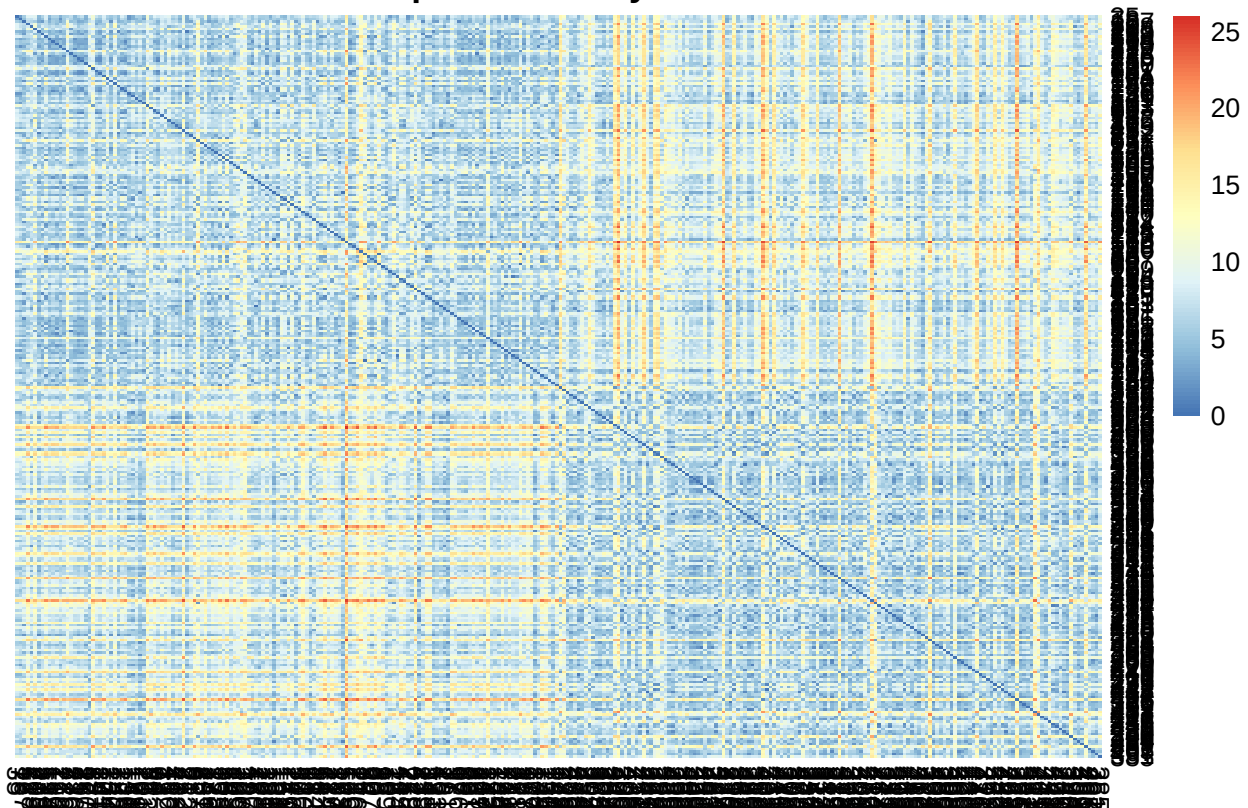
```
xdisseuc=daisy(standx, metric='euclidean', stand=FALSE)
xdissman=daisy(standx, metric='manhattan', stand=FALSE)
```

We can visualize them through the heatmaps:

```
pheatmap(as.matrix(xdisseuc), cluster_cols = FALSE, cluster_rows = FALSE, main='Heatmap Dissimilarity E
```

```
heatmap(as.matrix(xdissman), cluster_rows = FALSE, cluster_cols = FALSE, main='Heatmap Dissimilarity M
```

Heatmap Dissimilarity Manhattan



Discussion:

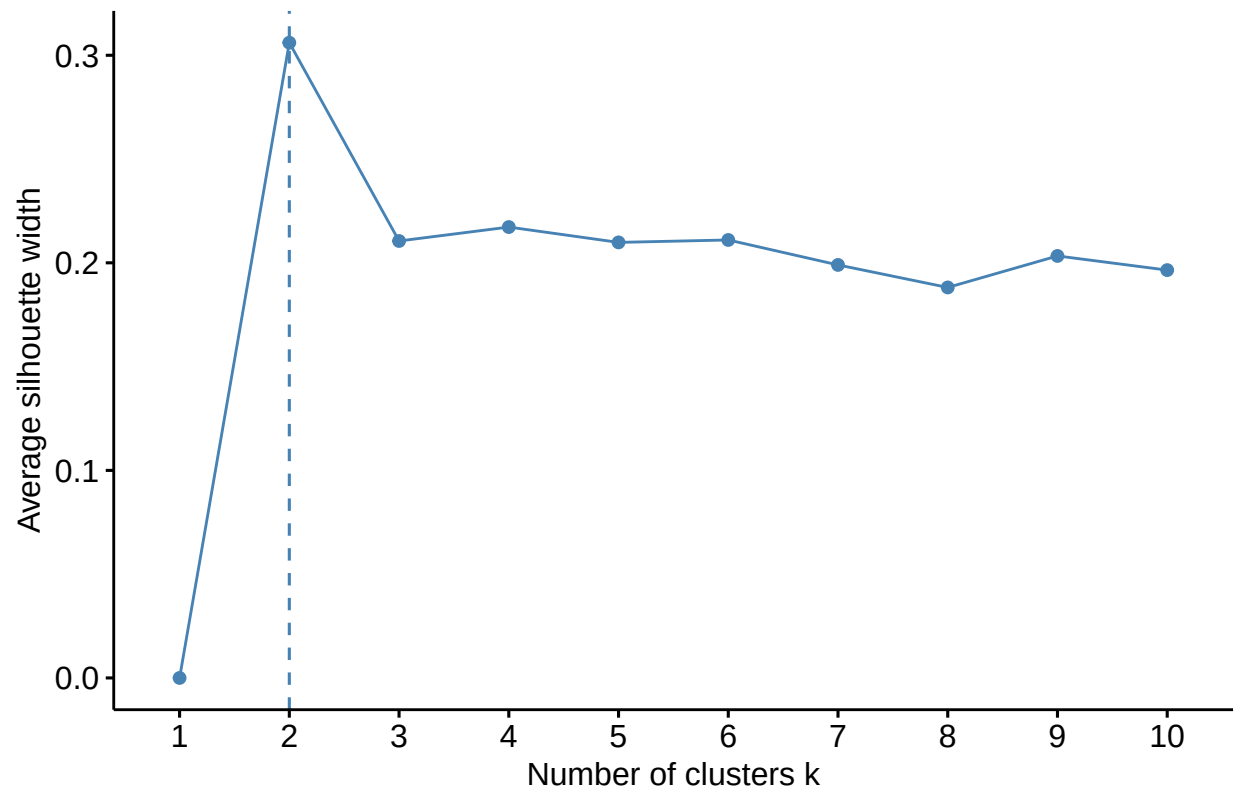
We can see a strong similarity in the patterns of the two heatmaps; indeed in terms of color distribution the two maps appear very similar. In addition, the blue blocks and orange ones are located in the same positions in both of them. Note that the blue areas represent highly similar snacks, while the orange/red areas indicate high dissimilarities, meaning they are quite different snacks. Because of this, we can assume that the cluster structure is robust. This implies that the groupings among the various products are strong enough to emerge clearly regardless of the distance metric chosen. We can conclude that the only relevant difference between the two is their scale. If we look at the legend in the heatmap, we can see that the maximum distance for the Manhattan metric is 25, whereas the Euclidean one stops at approximately 11. This makes sense as it is a consequence of the behavior of the two metrics: the Manhattan distance sums absolute differences across all variables, while the Euclidean calculates the square root of the sum of squares, leading to smaller numerical values.

Question 1.3

Now with the silhouette method determine the optimal number of cluster K^* . Then plot the silhouette plot.

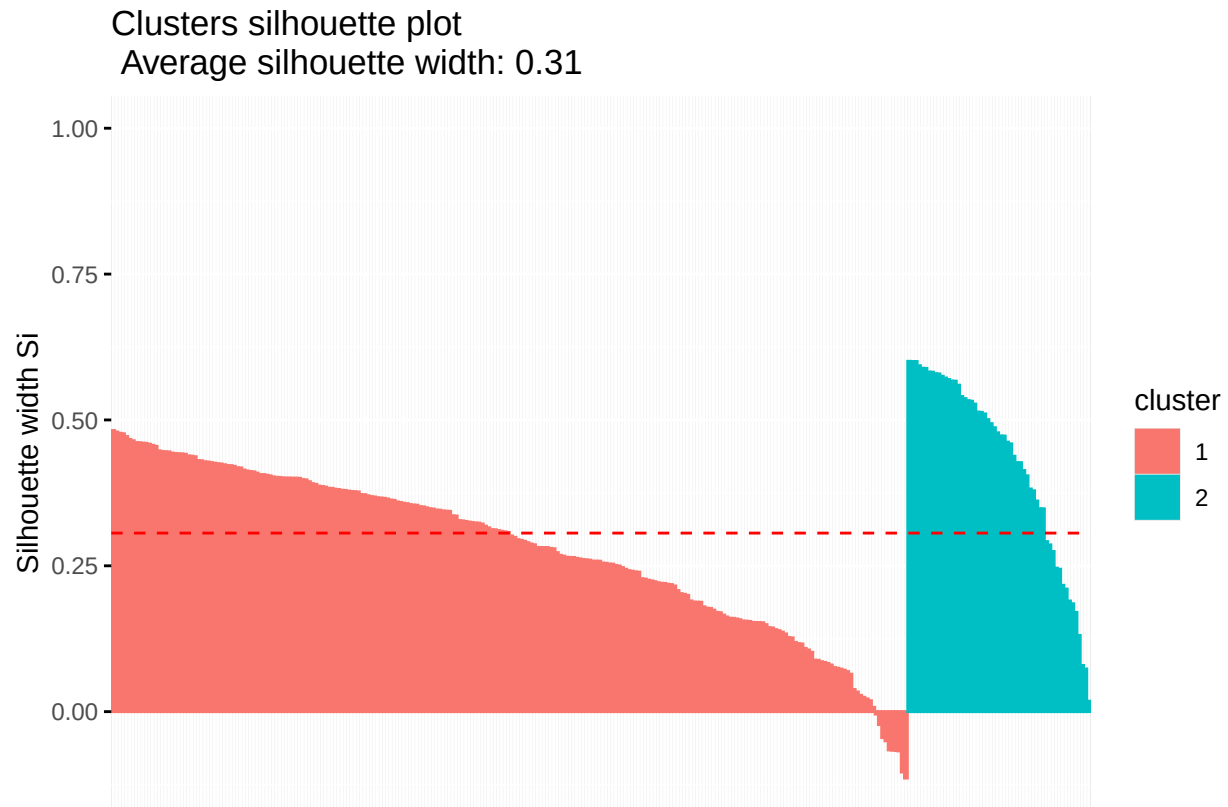
```
fviz_nbclust(standx,method='silhouette', cluster::pam)
```

Optimal number of clusters (method = "silhouette")



```
pams=pam(standx, k=2)
fviz_silhouette(pams)
```

```
##   cluster size ave.sil.width
## 1      1  244      0.28
## 2      2   56      0.43
```



Discussion:

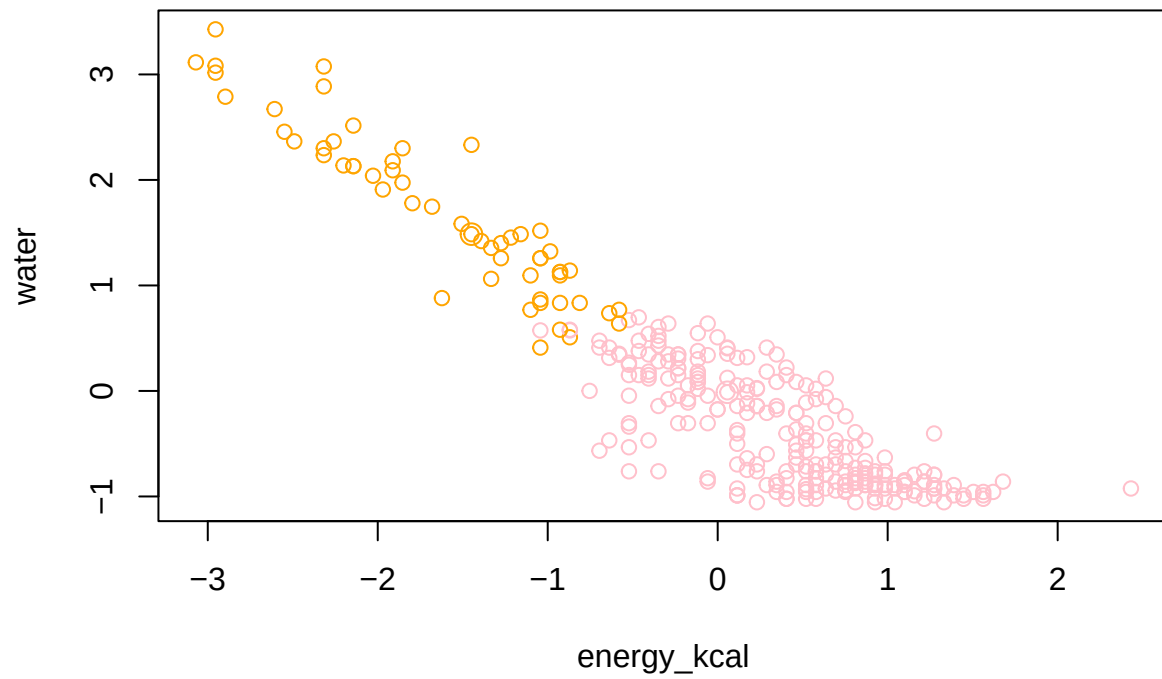
The ideal number of clusters is the one at which the silhouette function reaches its maximum. In this case, it is for $K=2$, therefore the optimal number of clusters for this dataset is determined to be $K^*=2$. From the silhouette plot we can see an average silhouette width of 0.31. This low value suggests a weak or overlapping cluster structure within the nutritional data. Furthermore, the data reveal an imbalance between the two clusters. The first cluster with 244 observations is less cohesive, showing an average width of 0.28. We can also spot some negative silhouette values in this cluster, which implies that several observations might be poorly classified. In contrast, the second cluster with only 56 observations, exhibits a better shape. It is more compact and also its average silhouette width is higher being equal to 0.43.

Question 1.4

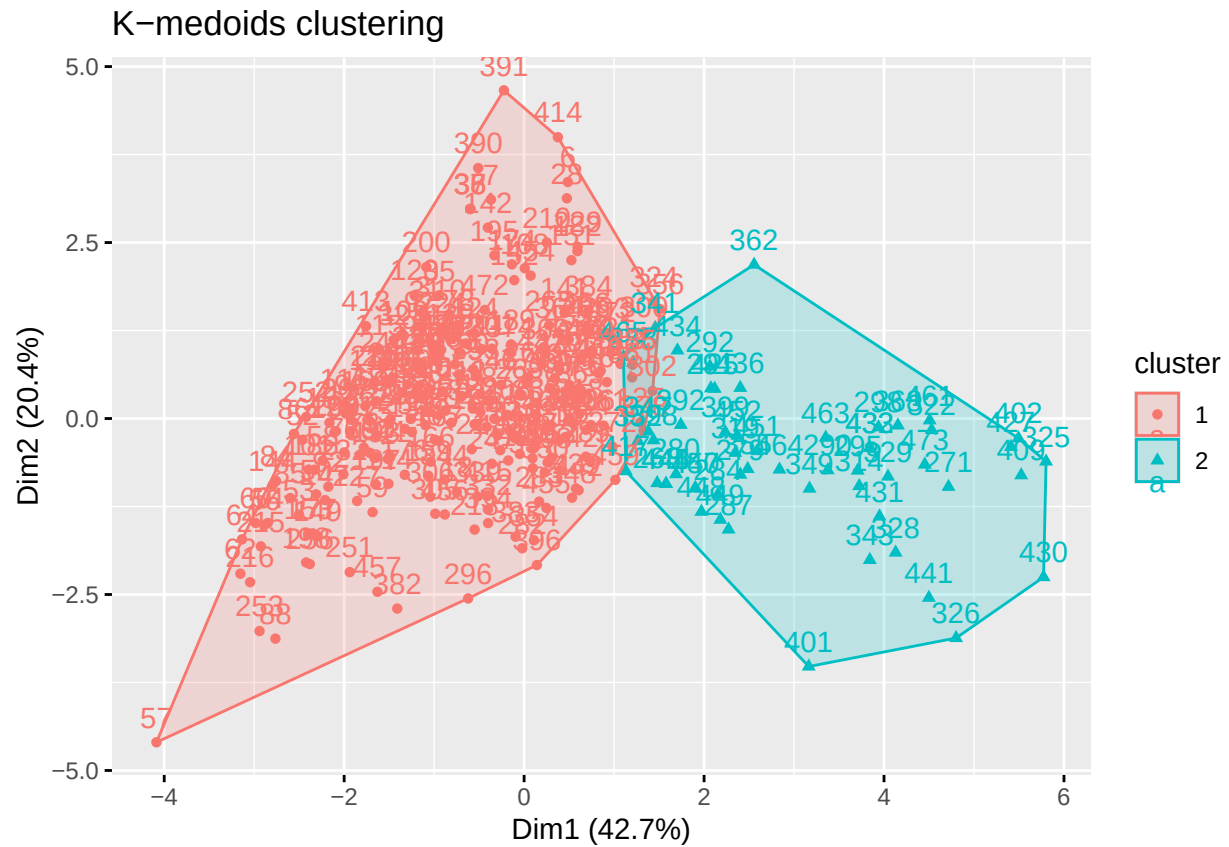
Now we perform a K-medoids clustering and hierarchical clustering (with average linkage). Then compare them with Rand Index or contingency matrix. We start with K-medoids clustering:

```
standxpam=pam(standx, k=2)
colors.obs=c('pink', 'orange')
plot(standx, col=colors.obs[standxpam$cluster], main='K-medoids')
points(standxpam$medoids, col=colors.obs, cex=1.5)
```

K-medoids



```
fviz_cluster(standxpam, data=standx, main = "K-medoids clustering")
```



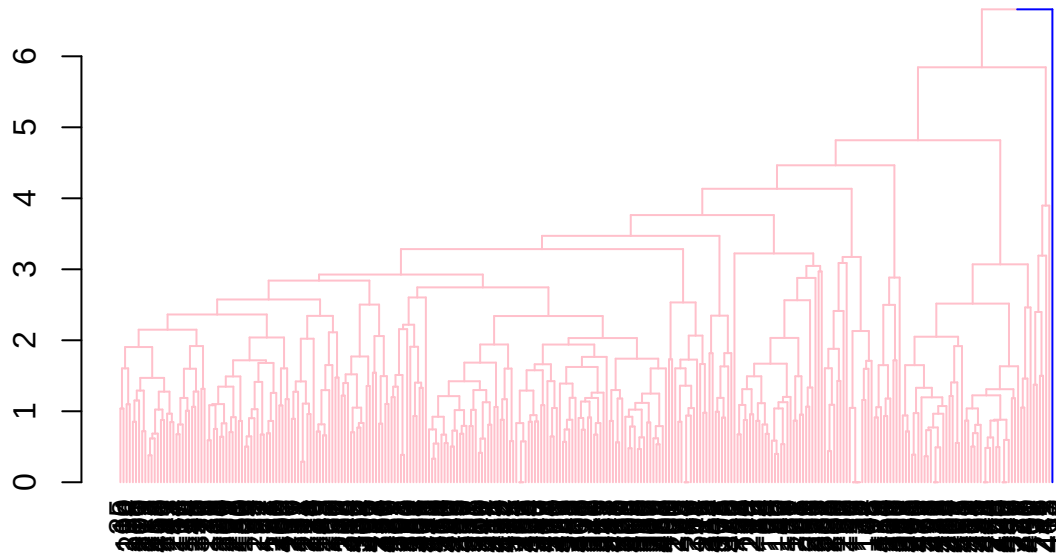
```
tabpam=table(True_labels=y_labels, PAM_cluster=standxpam$clustering )
corrplot(as.matrix(tabpam), is.corr=FALSE, method='color', addCoef.col = 'black', tl.col='black', addgr
```



We now perform the hierarchical clustering:

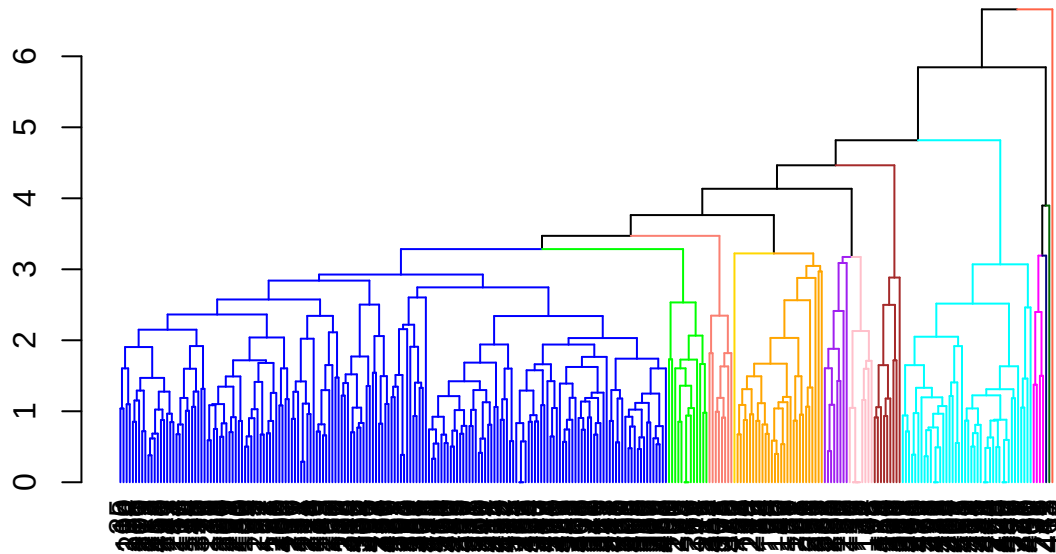
```
library(dendextend)
xstandagnesaverage=agnes(standx, method='average')
dend=as.dendrogram(xstandagnesaverage)
dendk2=color_branches(dend, k=2, col=c('pink', 'blue'))
plot(dendk2, main='Dendrogram k=2')
```

Dendrogram k=2



```
col13=c("blue", "green", 'salmon', "gold", "orange", "purple",
        "pink", "brown", "cyan", "magenta", "navy", "darkgreen", "tomato")
xstandagnesaverage=agnes(standx, method='average')
dend=as.dendrogram(xstandagnesaverage)
dendk13=color_branches(dend, k=13, col=col13)
plot(dendk13, main='Dendrogram k=13')
```

Dendrogram k=13



```
hcclusters=cutree(xstandagnesaverage, k=2)
```

Now we compare the two methods first with a contingency table and then with Rand Index:

```
table(standxpam$clustering, True_Labels=y_labels)
```

```
##      True_Labels
##      biscuits cakes
## 1         150     94
## 2           0     56
```

```
table(hcclusters, True_labels=y_labels)
```

```
##          True_labels
## hcclusters biscuits cakes
##          1         149    150
##          2           1      0
```

```
rand.index(standxpam$clustering, as.numeric(y_labels))
```

```
## [1] 0.5682497
```

```
rand.index(hcclusters, as.numeric(y_labels))
```

```
## [1] 0.4983501
```

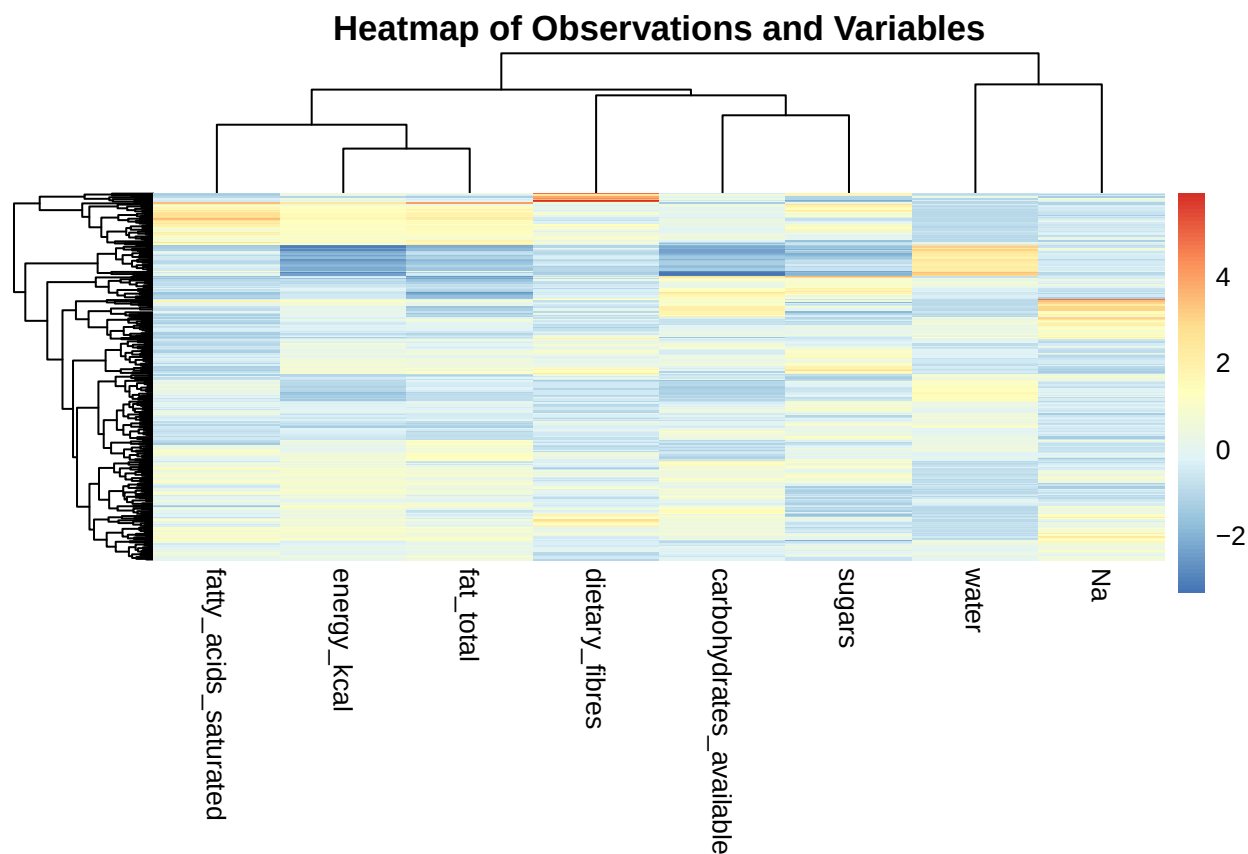
Discussion:

First, we can observe that the K-medoids(pam) method performs significantly better than the average linkage clustering in recovering the true product categories. Evidence of this is the rand index, indeed for pam is equal to 0.569, indicating a moderate agreement with the true label. Although it is not perfect, it shows that the algorithm identified successfully a structural pattern in data. For the hierarchical linkage the value is equal to 0.498, noting that in a dataset with two balanced categories a value of 0.5 is practically equivalent to random assignment. Then analyzing the contingency table, we can see that the pam method was able to identify two distinct groups. Specifically, Cluster 1 contains all 150 biscuits and 94 cakes, while cluster 2 consists only of the 56 remaining cakes. This suggests that pam is able to isolate a specific nutritional profile: Cluster 2 has higher water content and lower energy density while Cluster 1 drier and more dense calories snacks. In addition, we can also notice that biscuits are nutritionally homogeneous, while cakes are more varied. In contrast, the agnes method was not able to separate the categories with K=2. It placed almost every observation in Cluster 1, leaving only one observation in Cluster 2. This implies that this method is not suitable for the dataset as it suffers from sensitivity to outliers or a lack of clear separations between categories. To explain why this happened we plotted the dendrogram cut at an arbitrary higher number of clusters (K=13, note that this new value was chosen as a diagnostic tool and not as a new optimal K), seeing that this plots illustrates the chaining phenomenon. Indeed instead of forming balanced cluster, the average linkage sequentially isolates individual extreme outliers or micro-clusters, placing the majority of the data in one huge block. For these reasons, we can conclude that the hierarchical method is too sensitive to outliers and is affected by the chaining effect, while the K-medoids, since more robust, is the most appropriate method for the analysis.

Question 1.5

Generate a heatmap with both observations and variable clustered. Then discuss whether specific clusters of snacks are related to high levels of particular nutrients.

```
pheatmap(standx, clustering_distance_rows = 'euclidean', clustering_distance_cols = 'euclidean', method=
```



Discussion:

We begin by studying the heatmap of observations and variables. The column dendrogram shows a strong positive correlation among specific nutrients. We can notice how “energy_kcal”, “fat_total” and “fatty_acids_saturated” cluster together, implying that the caloric density of these snacks is highly driven by fat content. On the other hand, “carbohydrates_available” and “sugars” form another branch. Conversely, “water” and “Na” are isolated on the right side, this shows that moisture content behaves independently to the energy dense macronutrients. For the observations, we can identify two main clusters. The first one is characterized by high energy and low moisture. These snacks show warm colors for fats, energy and carbohydrates, while showing cool colors for water (noting that warm colors represent positive scaled values meanwhile cooler colors represent negative scaled values). In contrast, the second cluster is the one with high moisture and low energy, for these observations the water column is orange, while the energy and fat columns are blue. Finally, certain observations display red peaks for “dietary_fibres” and “Na”. These are snacks with a unique nutritional composition that deviates from the two primary profiles. We can conclude that specific clusters of snacks are indeed related to high levels of particular nutrients.

Part 2: Multiple Linear Regression

Loading Data and Setup

```
set.seed(1085999) # Replace it with your student number
```

```

train_idx_biscuits <- sample(which(mydatafull$labels == "biscuits"), size = 100)
train_idx_cakes <- sample(which(mydatafull$labels == "cakes"), size = 100)
train_idx <- c(train_idx_biscuits, train_idx_cakes)

mytrainingdata <- mydatafull[train_idx, ]
myvalidationdata <- mydatafull[-train_idx, ]

```

Question 2.1

We start by performing an exploratory data analysis on mytrainingdata:

```
dim(mytrainingdata)
```

```
## [1] 200 9
```

```
head(mytrainingdata)
```

```

##      energy_kcal water carbohydrates_available sugars dietary_fibres fat_total
## 242          475   1.0                56    16          13.0          21
## 161          400  21.0                55    22           2.0          17
## 247          470  13.5                53    22           2.5          25
## 257          480   2.0                63    28           7.0          20
## 44           500   8.5                50    47           4.5          28
## 251          565   2.0                52    17           3.0          35
##      fatty_acids_saturated  Na  labels
## 242                6.0 0.12 biscuits
## 161                7.0 0.08 biscuits
## 247                9.0 0.18 biscuits
## 257                9.0 0.10 biscuits
## 44                 2.5 0.03 biscuits
## 251               14.0 0.01 biscuits

```

```
summary(mytrainingdata)
```

```

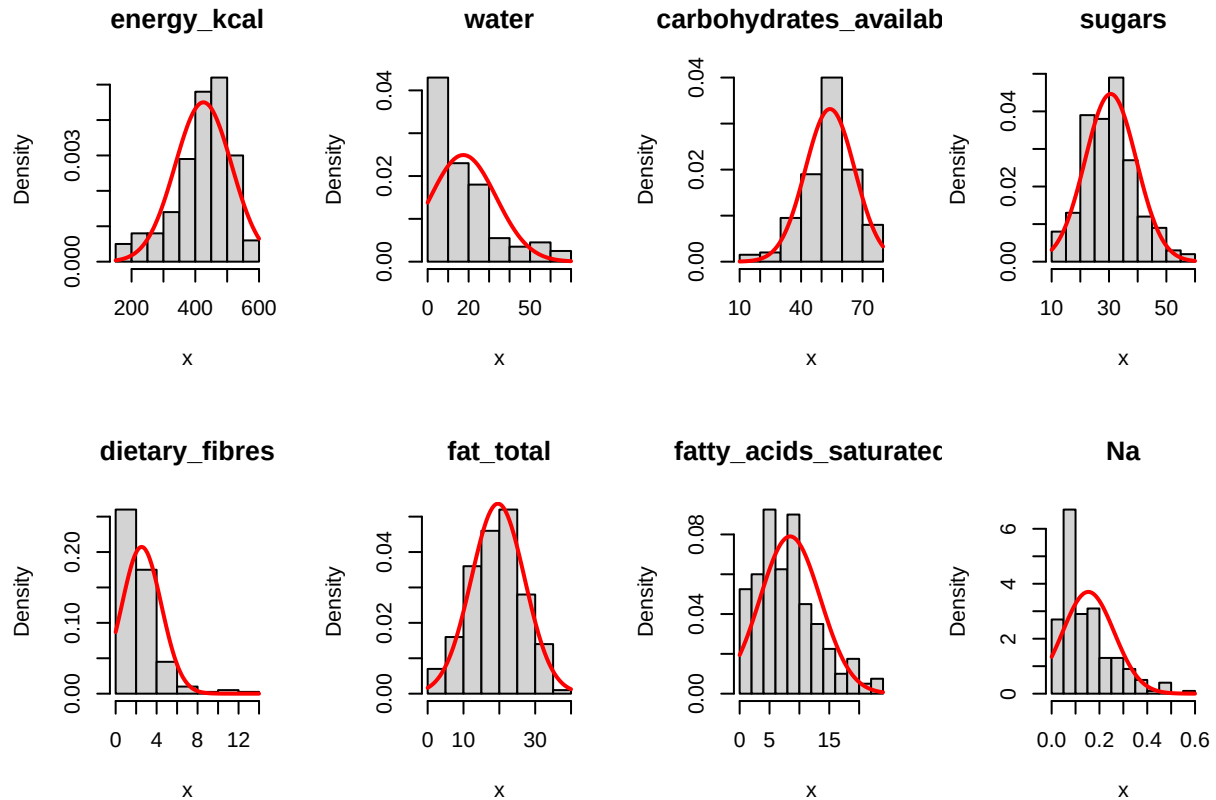
##      energy_kcal      water carbohydrates_available      sugars
## Min.   :165.0   Min.   : 0.50   Min.   :15.80         Min.   :11.00
## 1st Qu.:385.0   1st Qu.: 4.00   1st Qu.:48.00         1st Qu.:24.00
## Median :442.5   Median :14.50   Median :55.00         Median :30.65
## Mean   :425.4   Mean   :17.41   Mean   :54.12         Mean   :30.60
## 3rd Qu.:486.2   3rd Qu.:24.00   3rd Qu.:61.00         3rd Qu.:36.00
## Max.   :575.0   Max.   :69.30   Max.   :80.00         Max.   :56.00
## dietary_fibres    fat_total    fatty_acids_saturated      Na
## Min.   : 0.000   Min.   : 2.00   Min.   : 0.600       Min.   :0.0000
## 1st Qu.: 1.500   1st Qu.:14.00   1st Qu.: 5.000       1st Qu.:0.0800
## Median : 2.000   Median :19.00   Median : 8.000       Median :0.1150
## Mean   : 2.537   Mean   :19.62   Mean   : 8.457       Mean   :0.1536
## 3rd Qu.: 3.000   3rd Qu.:25.00   3rd Qu.:11.000       3rd Qu.:0.2000
## Max.   :13.000   Max.   :38.00   Max.   :24.000       Max.   :0.5510
##      labels
## biscuits:100

```

```
## cakes :100
##
##
##
##
```

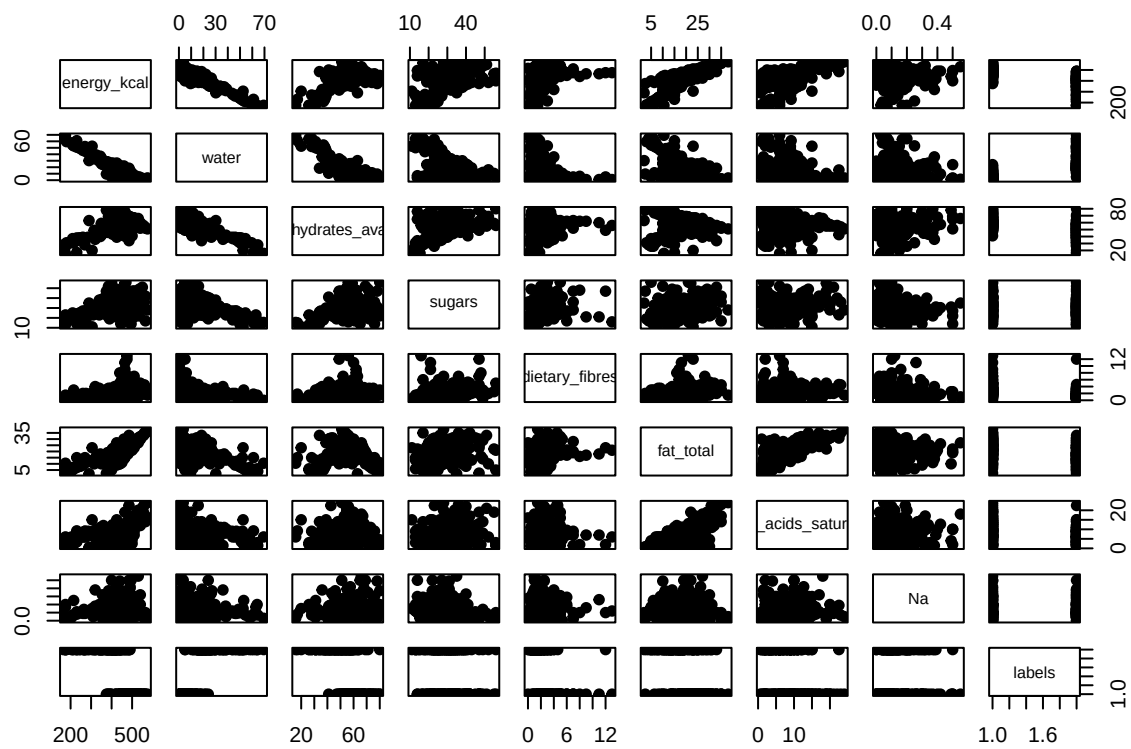
We now create some plots to have a better view:

```
par(mfrow=c(2,4))
for(i in 1:8){
  x=mytrainingdata[,i]
  hist(x, freq=FALSE,main=colnames(mytrainingdata)[i])
  mn=mean(x,na.rm=TRUE)
  std=sd(x, na.rm=TRUE)
  curve(dnorm(x,mean=mn,sd=std),col='red',lwd=2,add=TRUE)
}
```

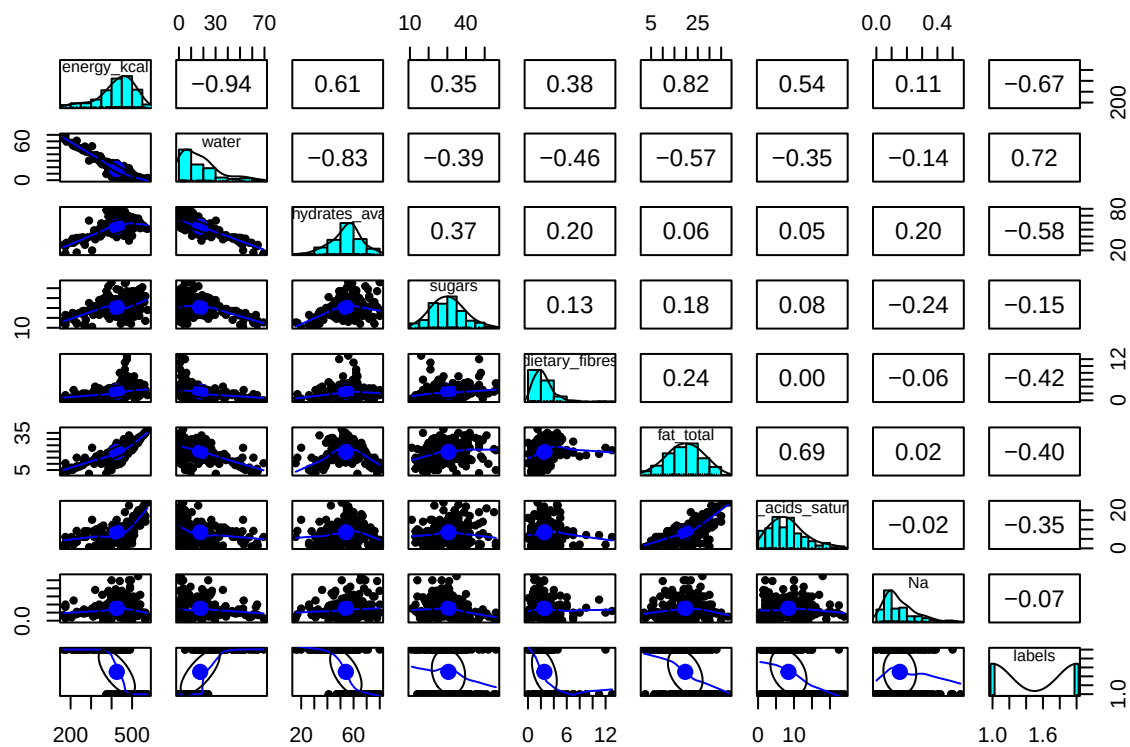


We now plot the pairs:

```
pairs(mytrainingdata, pch=19)
```

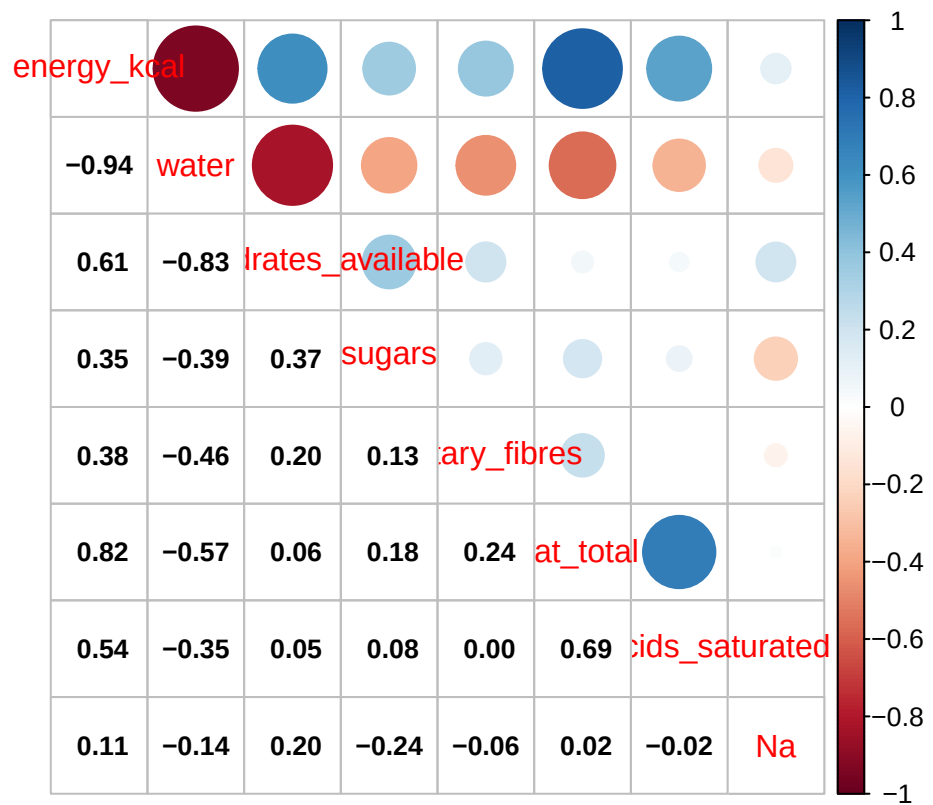


```
pairs.panels(mytrainingdata)
```



For the correlation we consider only the numeric ones:

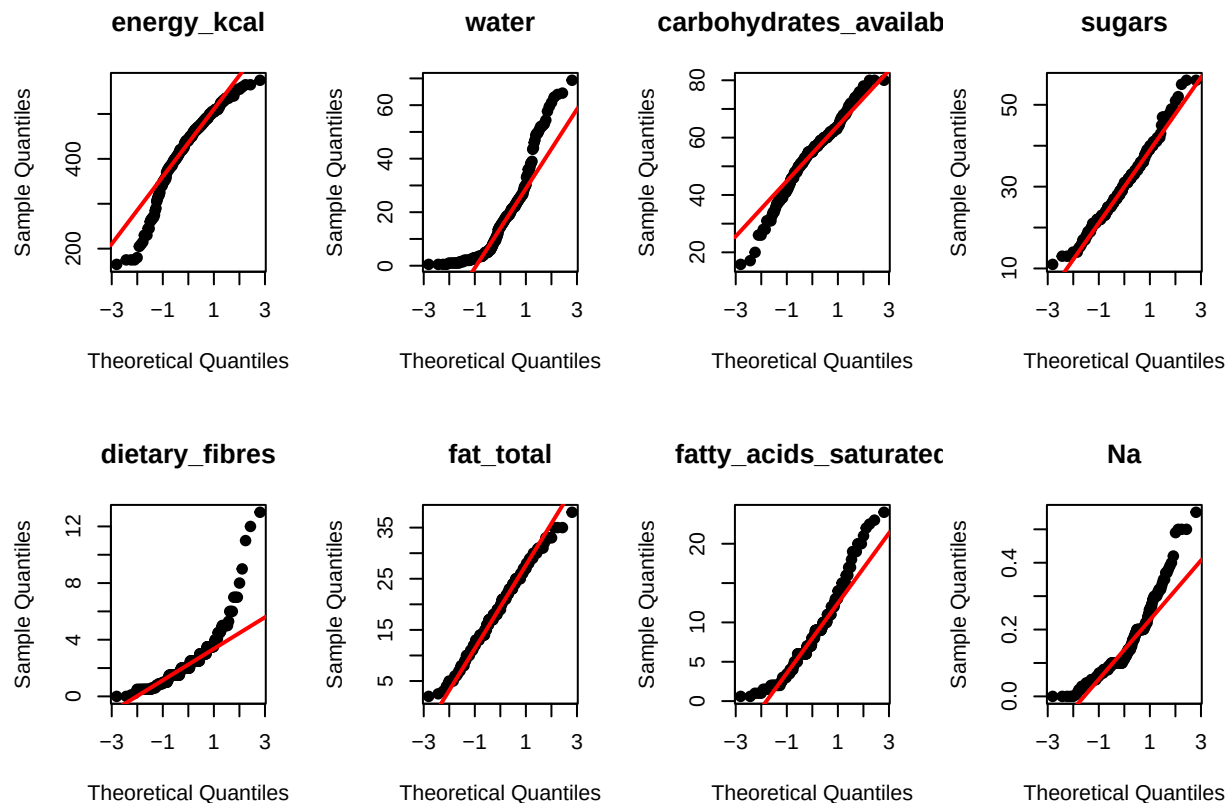
```
numtraining=mytrainingdata[,sapply(mytrainingdata, is.numeric)]
mytrainingdatacor=cor(numtraining)
corrplot.mixed(mytrainingdatacor, number.cex=0.8, lower.col='black')
```



```

par(mfrow=c(2,4))
for(i in 1:8){
qqnorm(mytrainingdata[,i],pch=19,main=colnames(mytrainingdata)[i])
qqline(mytrainingdata[,i],col='red',lwd=2)
}

```



We check the relationship between “energy_kcal” and the predictors.

```
for (i in 1:8){
  print(skewness(mytrainingdata[,i]))
}
```

```
## [1] -0.9771505
## [1] 1.223151
## [1] -0.505381
## [1] 0.3710139
## [1] 2.338429
## [1] -0.07717119
## [1] 0.7945097
## [1] 1.260796
```

I have to transform the variables with a skewness bigger than 1 or smaller than -1, so the second one, fifth one and eighth one. We now define them and try to find the best transformation to have a skewness closer to zero:

```
water=mytrainingdata[,2]
dietary=mytrainingdata[,5]
Na=mytrainingdata[,8]
```

We start with the transformations for water:

```
sqrwater=sqrt(water)
skewness(sqrwater)
```

```
## [1] 0.4031652
```

```
cubicwater=(water)^(1/3)
skewness(cubicwater)
```

```
## [1] 0.1094695
```

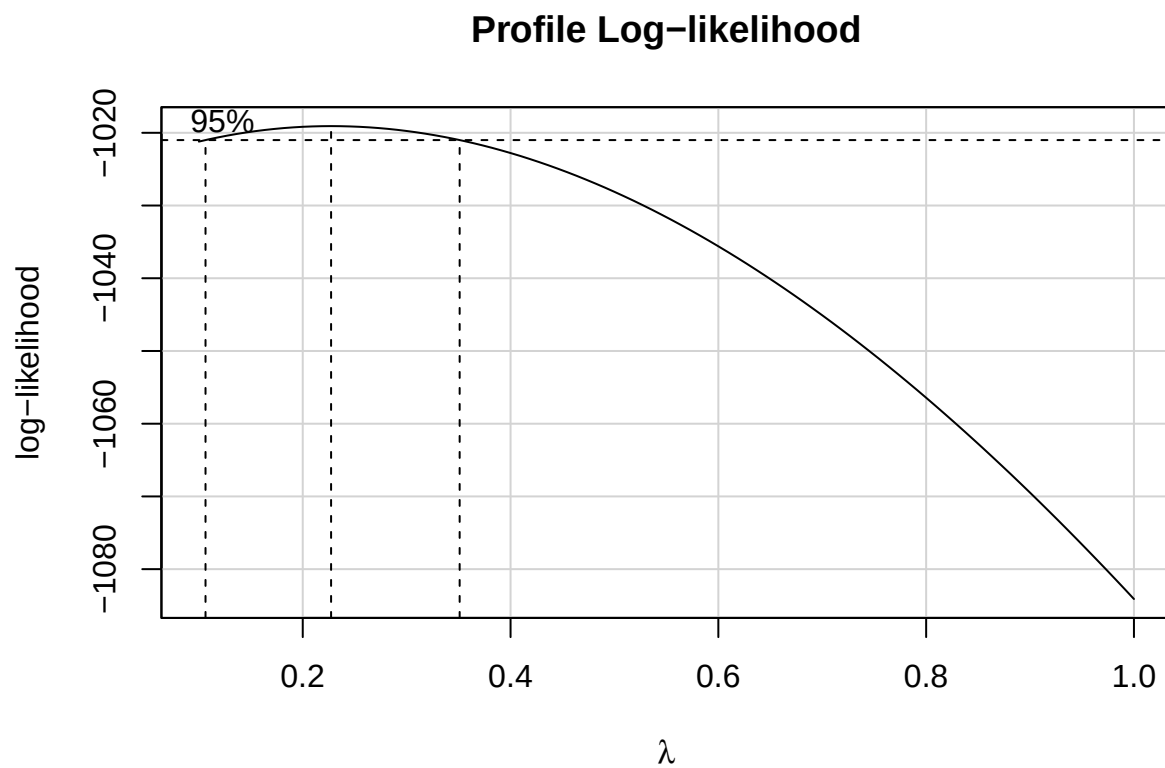
```
invwater=1/(water)
skewness(invwater)
```

```
## [1] 3.531033
```

```
logwater=log(water)
skewness(logwater)
```

```
## [1] -0.5304357
```

```
boxcoxwater=boxCox(water~1,seq(0.1,1,0.1))
```



```
lambda=boxcoxwater$x[which.max(boxcoxwater$y)]
boxwater=bcPower(water, lambda)
skewness(boxwater) #the best one->closest to 0 skewness
```

```
## [1] -0.08408035
```

We now do the same with dietary:

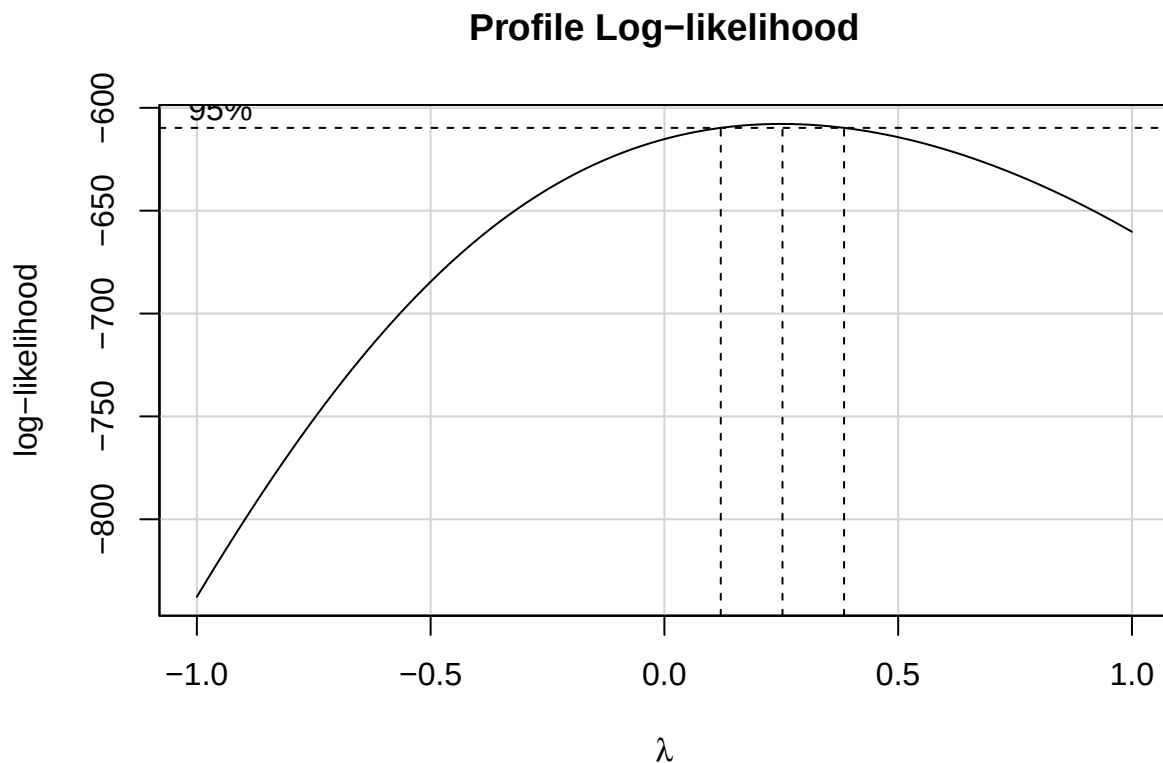
```
sqrtdiet=sqrt(dietary)
skewness(sqrtdiet)
```

```
## [1] 0.6653345
```

```
cubicdiet=(dietary)^(1/3)
skewness(cubicdiet)
```

```
## [1] -0.1720486
```

```
boxcoxdiet=boxCox((0.1+dietary)~1,seq(-1,1,0.1))
```



```
lambda=boxcoxdiet$x[which.max(boxcoxdiet$y)]
boxdiet=bcPower((0.1+dietary), lambda)
skewness(boxdiet) #the best one->closest to 0 skewness
```

```
## [1] 0.06511713
```

We do the same with the Na:

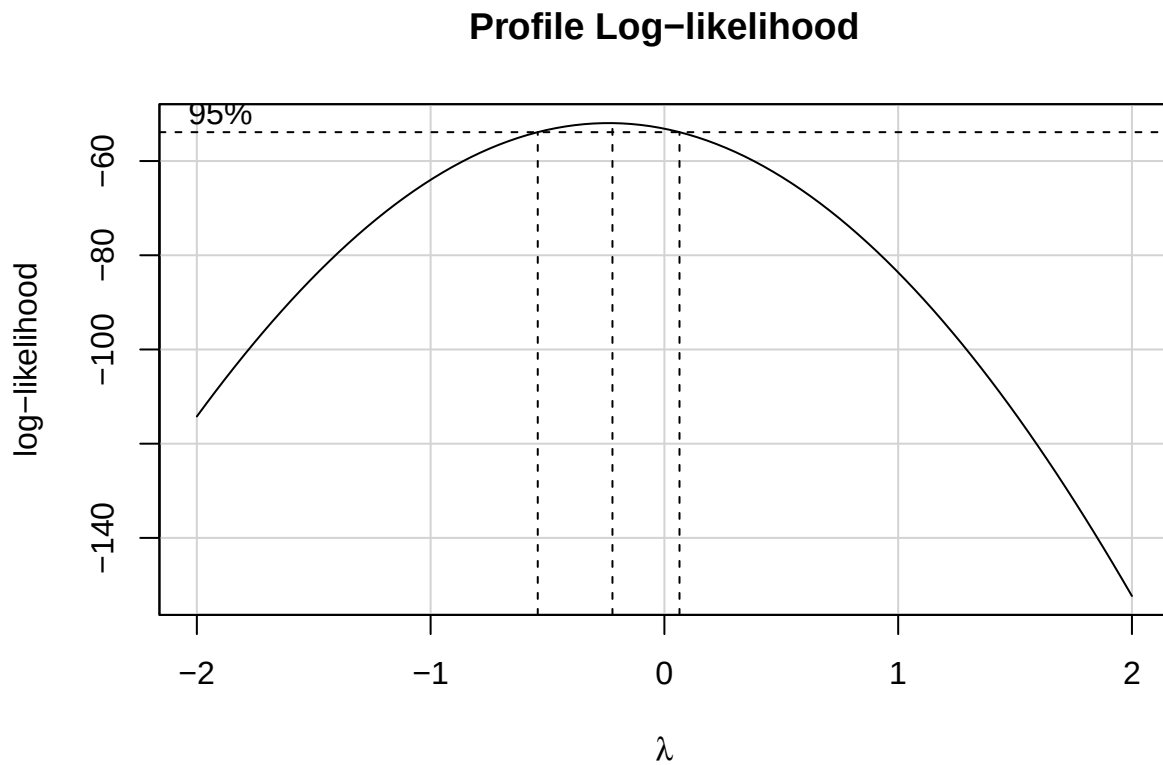
```
sqrtna=sqrt(Na)
skewness(sqrtna)
```

```
## [1] 0.0889104
```

```
cubicna=(Na)^(1/3)
skewness(cubicna)
```

```
## [1] -0.8428259
```

```
boxcoxna=boxCox((0.1+Na)~1,seq(-2,2,0.1))
```



```
lambda=boxcoxna$x[which.max(boxcoxna$y)]
boxna=bcPower((0.1+Na), lambda)
skewness(boxna)
```

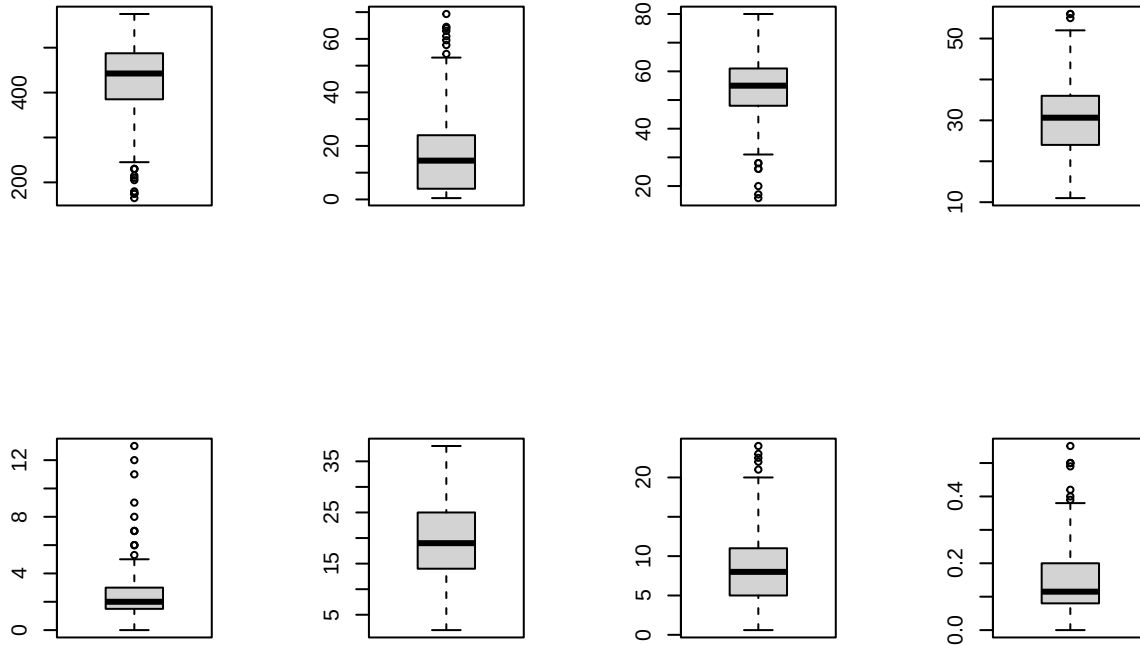
```
## [1] 0.02452138
```

With the boxplot we try to see if there are any outliers:

```

par(mfrow=c(2,4))
for (i in 1:8){
  boxplot(mytrainingdata[,i], horizontal=FALSE)
}

```



We check the normality with shapiro-wilk:

```

for (i in 1:8){
  print(shapiro.test(mytrainingdata[,i]))
}

```

```

##
##  Shapiro-Wilk normality test
##
## data:  mytrainingdata[, i]
## W = 0.92965, p-value = 3.165e-08
##
##
##  Shapiro-Wilk normality test
##
## data:  mytrainingdata[, i]
## W = 0.86316, p-value = 1.961e-12
##
##
##  Shapiro-Wilk normality test

```

```
##
## data: mytrainingdata[, i]
## W = 0.97652, p-value = 0.001962
##
##
## Shapiro-Wilk normality test
##
## data: mytrainingdata[, i]
## W = 0.98613, p-value = 0.04708
##
##
## Shapiro-Wilk normality test
##
## data: mytrainingdata[, i]
## W = 0.80423, p-value = 4.117e-15
##
##
## Shapiro-Wilk normality test
##
## data: mytrainingdata[, i]
## W = 0.99272, p-value = 0.4247
##
##
## Shapiro-Wilk normality test
##
## data: mytrainingdata[, i]
## W = 0.94725, p-value = 1.033e-06
##
##
## Shapiro-Wilk normality test
##
## data: mytrainingdata[, i]
## W = 0.89464, p-value = 1.16e-10
```

Only one of the variable is normal.

With `mytrainingdatanew` we create the new dataset with the transformed data:

```
mytrainingdatanew=mytrainingdata
mytrainingdatanew$water=boxwater
mytrainingdatanew$dietary_fibres=boxdiet
mytrainingdatanew$Na=boxna
```

Discussion:

We begin with an exploratory data analysis. First, the summary function reveals that the variables are measured on very different scales and most deviate from a normal distribution. Indeed, both the QQ-plots and the Shapiro-Wilk test indicate that the only normal distributed variable is “fat_total”. Next, we examine the histograms, QQ-plots and skewness values to discover the direction of the skew. The histograms of “water”, “dietary_fibres” and “Na” indicate strong right skewness, this is confirmed by the skewness values which are all greater than 1. The boxplots also confirm the right-skewness, as they display long right tails and upper outliers. Conversely, “energy_kcal” histogram looks strongly left-skewed, a proof is the skewness value close to -1. We evaluated the best transformations for the variables with a skewness >1 or <-1 (“water”, “dietary_fibres” and “Na”). We tested three types of transformations: square root, cubic and boxcox.

For “water”, we also tested inverse and logarithmic transformations but they were ineffective. We chose the following transformations:

```
-“water”: boxcox (new skewness=-0.08032888);
-“dietary_fibres”: boxcox (new skewness=0.06835412);
-“Na”: boxcox (new skewness=0.024);
```

Then, we analyzed the correlation matrix and pairs panels to check for linear relationships and multicollinearity. Regarding the target variable “energy_kcal”, it is strongly negatively correlated with “water”, while it shows a strong positive correlation with “fat_total” and “carbohydrates_available”. Furthermore, we observed strong correlation among the predictors themselves, such as a negative correlation between “water” and “carbohydrates_available”, and a strong positive correlation between “fat_total” and “acid_saturated”.

Question 2.2

Fit a full linear regression model using all available predictors. Report the coefficients and R^2 . Is the overall model significant?

```
linearfit=lm(energy_kcal~., data=mytrainingdatanew)
summary(linearfit)
```

```
##
## Call:
## lm(formula = energy_kcal ~ ., data = mytrainingdatanew)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
	-13.2337	-3.4645	-0.4966	3.1230	17.9594

```
##
## Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	59.13655	7.09092	8.340	1.45e-14 ***
water	-2.99644	0.64252	-4.664	5.83e-06 ***
carbohydrates_available	3.64854	0.07579	48.139	< 2e-16 ***
sugars	0.03026	0.05638	0.537	0.5921
dietary_fibres	6.41298	0.63958	10.027	< 2e-16 ***
fat_total	9.04823	0.09730	92.989	< 2e-16 ***
fatty_acids_saturated	-0.72564	0.12152	-5.971	1.13e-08 ***
Na	-1.15294	0.83738	-1.377	0.1702
labelscakes	-2.46216	1.41818	-1.736	0.0842 .

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.844 on 191 degrees of freedom
## Multiple R-squared:  0.9958, Adjusted R-squared:  0.9957
## F-statistic: 5701 on 8 and 191 DF, p-value: < 2.2e-16
```

Discussion:

We start by examining the R^2 value, which is equal to 0.9958 (with an adjusted R^2 of 0.9957). This indicates an excellent goodness of fit meaning that the 99,58% of the variance in “energy_kcal” is explained by the predictors. Next, we look at the F-statistics, which is equal to 5701 on 8 and 191 degrees of freedom, yielding a p-value<2.2e-16 (well below the significance level of 0.05). We can conclude that the overall model is highly significant. We proceed with the coefficient analysis, we can see

that macronutrients like “fat_total”(estimate=9.04), “carbohydrates_available”(estimate=3.64) and “dietary_fibres”(estimate=6.41) have strong positive coefficients. Furthermore, they are highly significant ($p < 0.001$), which is expected since they are the primary drivers of the snack caloric density. Conversely, “water” has a strongly negative coefficient (-2.99), confirming that higher moisture decreases caloric density. Finally, it is worth noting that “Na” and “sugars” are not statistically significant in the full model ($p > 0.05$).

Question 2.3

Check for multicollinearity among the predictors using the Variance Inflation Factor. If $VIF > 10$ suggest a strategy to handle it and re-fit a simplified model.

```
library(DAAG)
vifdata=DAAG::vif(linearfit)
summary(linearfit)
```

```
##
## Call:
## lm(formula = energy_kcal ~ ., data = mytrainingdatanew)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.2337  -3.4645  -0.4966   3.1230  17.9594
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    59.13655     7.09092   8.340 1.45e-14 ***
## water         -2.99644     0.64252  -4.664 5.83e-06 ***
## carbohydrates_available  3.64854     0.07579  48.139 < 2e-16 ***
## sugars         0.03026     0.05638   0.537  0.5921
## dietary_fibres  6.41298     0.63958  10.027 < 2e-16 ***
## fat_total      9.04823     0.09730  92.989 < 2e-16 ***
## fatty_acids_saturated -0.72564     0.12152  -5.971 1.13e-08 ***
## Na            -1.15294     0.83738  -1.377  0.1702
## labelscakes    -2.46216     1.41818  -1.736  0.0842 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.844 on 191 degrees of freedom
## Multiple R-squared:  0.9958, Adjusted R-squared:  0.9957
## F-statistic: 5701 on 8 and 191 DF, p-value: < 2.2e-16
```

```
R=summary(linearfit)$r.squared
vifdata
```

```
##              water carbohydrates_available              sugars
##              8.8949              4.8435              1.4791
## dietary_fibres              fat_total  fatty_acids_saturated
##              1.7477              3.0486              2.1938
##              Na              labelscakes
##              1.2007              2.9449
```

R

```
## [1] 0.9958294
```

Discussion:

We performed a multicollinearity check using the Variance Inflation Factor(VIF). As shown in the output, all the VIF values are below the critical threshold of 10 although the highest is “water” at 8.894 quite close to 10 but still smaller. Because no variable is larger than 10 there is no problematic multicollinearity in our model. So we don’t need to apply any mitigation strategy like dropping highly correlated variables or refit a simplified model.

Then, we also extract R^2 which is equal to 0.9958. This is a confirmation that our full model is able to explain successfully the 99.58% of the variance in the snacks’ energy content.

Question 2.4

Perform a model selection using forward selection based on AIC. Report the selected model and compare it with the full model using ANOVA test. Which variable are retained in the simplified model?

```
lmfull=lm(energy_kcal ~ ., data=mytrainingdatanew)
lminitial=lm(energy_kcal ~ 1, data=mytrainingdatanew)
stepmodel=stepAIC(lminitial, scope=list(upper=lmfull, lower=lminitial), direction='forward', trace=FALSE)
anovacomp=anova(stepmodel, lmfull)
print(anovacomp)
```

```
## Analysis of Variance Table
##
## Model 1: energy_kcal ~ water + fat_total + carbohydrates_available + dietary_fibres +
##      fatty_acids_saturated + labels + Na
## Model 2: energy_kcal ~ water + carbohydrates_available + sugars + dietary_fibres +
##      fat_total + fatty_acids_saturated + Na + labels
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1      192 6532.1
## 2      191 6522.3  1    9.8338 0.288 0.5921
```

```
summary(stepmodel)
```

```
##
## Call:
## lm(formula = energy_kcal ~ water + fat_total + carbohydrates_available +
##      dietary_fibres + fatty_acids_saturated + labels + Na, data = mytrainingdatanew)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.4447  -3.3638  -0.5765   3.0898  18.1280
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    58.27348     6.89331   8.454 7.00e-15 ***
## water         -2.93510     0.63110  -4.651 6.14e-06 ***
```

```
## fat_total          9.06239      0.09348  96.941 < 2e-16 ***
## carbohydrates_available 3.66597      0.06835  53.638 < 2e-16 ***
## dietary_fibres      6.45786      0.63291  10.203 < 2e-16 ***
## fatty_acids_saturated -0.72557      0.12130  -5.982 1.06e-08 ***
## labelscakes        -2.37724      1.40671  -1.690  0.0927 .
## Na                 -1.31483      0.77970  -1.686  0.0934 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.833 on 192 degrees of freedom
## Multiple R-squared:  0.9958, Adjusted R-squared:  0.9957
## F-statistic: 6539 on 7 and 192 DF, p-value: < 2.2e-16
```

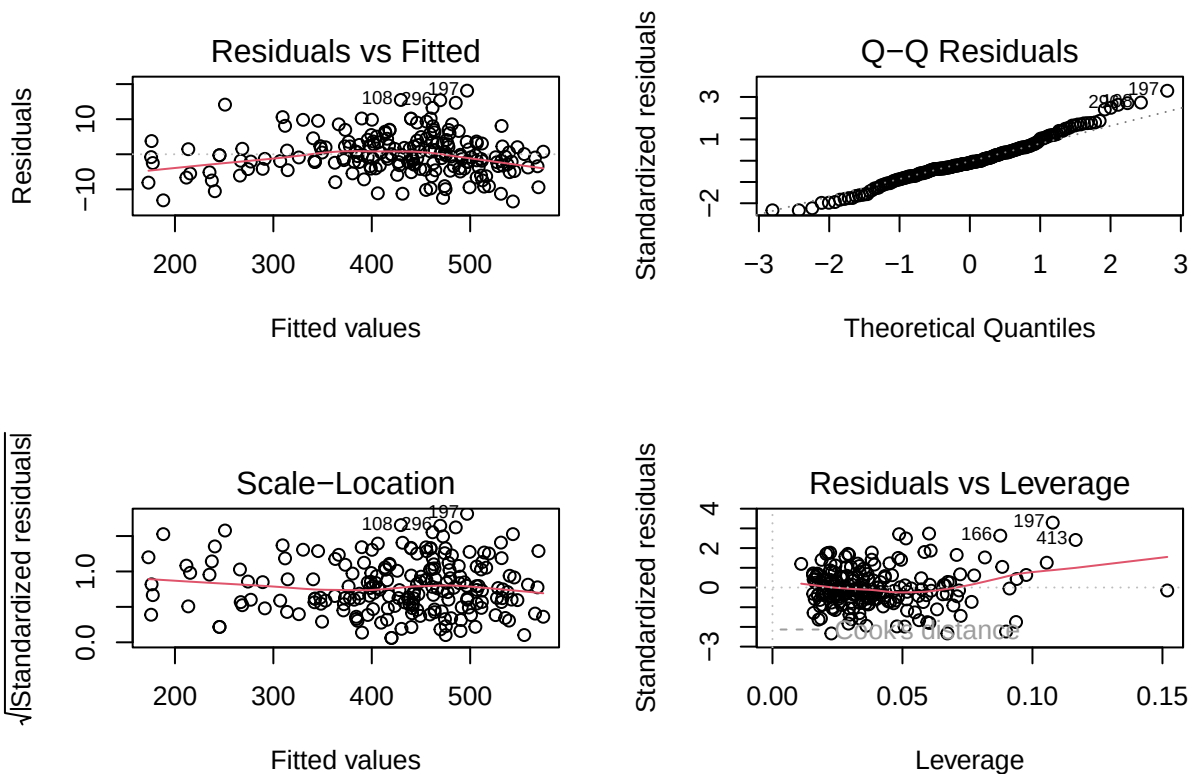
Discussion:

We began the model selection via forward selection based on the AIC(Akaike Information Criterion). Starting with a null model, predictors were added sequentially to maximize the reduction in AIC. Ultimately, we retained seven variables: “water”, “fat_total”, “carbohydrates_available”, “dietary_fibres”, “fatty_acids_saturated”, “Na” and “labels”. The variable “sugars” was excluded from the model. Indeed, adding “sugars” at the final step would have increased the AIC. This implies that it does not provide sufficient additional explanatory power to justify the increased model complexity. Then, we conducted a model comparison between the simplified model and the full one using an ANOVA test. We found an F-statistic of 0.288 and a p-value of 0.592. Since the p-value is greater than 0.05, there are no statistically significant differences in performance between the two models, confirming that the simplified model is preferable.

Question 2.5

Perform a comprehensive residual analysis of the final model. Check for normality homoscedasticity, and independence errors. Identify any influential observations and discuss their potential impact.

```
par(mfrow=c(2,2))
plot(stepmodel)
```



```
shapiro.test(residuals(stepmodel))
```

```
##
## Shapiro-Wilk normality test
##
## data: residuals(stepmodel)
## W = 0.98332, p-value = 0.01799
```

```
ncvTest(stepmodel)
```

```
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values
## Chisquare = 0.002112536, Df = 1, p = 0.96334
```

```
durbinWatsonTest(stepmodel)
```

```
## lag Autocorrelation D-W Statistic p-value
## 1 0.02721008 1.930822 0.576
## Alternative hypothesis: rho != 0
```

We now check for outliers doing an outliers detection:

```

modelstudres = studres(stepmodel)
outliersy=which(abs(modelstudres)>qnorm(0.995))
print(outliersy)

```

```

## 166 108 197 296
## 23 46 62 186

```

Discussion:

First, we evaluate the normality of the errors. The Shapiro-Wilk yields a p-value of 0.01799 (< 0.05), indicating a non-normal distribution. However, looking at the QQ-plot, the residuals align very tightly along the theoretical diagonal (with only small deviations at the extreme right tail). It is important to note that, given the relatively large sample size ($n=200$), the test is highly sensitive to minor tail fluctuations. Regarding homoscedasticity, the non-constant variance score test yields a p-value of 0.963, meaning we fail to reject the null hypothesis. This conclusion is supported by the Residuals vs Fitted and Scale Location plots: the points are randomly scattered and the smoothing lines remain nearly perfectly flat. Then through Durbin Watson we were able to show that residuals are independent, indeed since DW statistic is close to 2 and the p-value is larger than 0.05 we can accept the null hypothesis. So there is no first order autocorrelation and the residuals are independent, satisfying the assumption of the linear regression model. Finally, we can study the presence of outliers and influential points. To do so we calculated the studentized residuals and applied a threshold at $qnorm(0.995)$. This found 8 anomalous observations (23, 62, 46, 62, 108, 166, 186, 197 and 296) which are the snacks whose energy content deviates from the model's prediction. It's important to notice that an outliers does not necessarily compromise the model unless it is highly influential. Indeed looking at the Residuals vs Leverage plot, we can see that these points (46, 62 and 186) have low leverage values. As a consequence none of these cross the critical Cook's distance boundaries (usually 0.5). This means that although they have unusual caloric values compared to macronutrient profiles, they are not influential. In conclusion they don't distort the estimated regression coefficient and the model remains robust.

Question 2.6

Estimate the prediction performance of your final model using myvalidationdata. Use RMSE test:

```

lambdawater=boxcoxwater$x[which.max(boxcoxwater$y)]
lambdadiet=boxcoxdiet$x[which.max(boxcoxdiet$y)]
lambdana=boxcoxna$x[which.max(boxcoxna$y)]
myvalidationdatanew=myvalidationdata
myvalidationdatanew$water=bcPower(myvalidationdata$water, lambdawater)
myvalidationdatanew$dietary_fibres=bcPower((0.1 + myvalidationdata$dietary_fibres), lambdadiet)
myvalidationdatanew$Na=bcPower((0.1+myvalidationdata$Na), lambdana)

```

```

library(caret)
library(lattice)
pred=predict(stepmodel, newdata=myvalidationdatanew)
nv=nrow(myvalidationdatanew)
rmsepval=sqrt(mean((myvalidationdatanew$energy_kcal-pred)^2))
print(rmsepval)

```

```

## [1] 5.707486

```

```
#RMSE(myvalidationdatanew$energy_kcal,pred) another possible way to calculate it
```

Discussion:

After applying the necessary transformations to the validation set, we obtained a RMSE of 5.7. This means that, on average, the model's predictions deviate from the true energy content by only 5.71 kcal per 100g. Considering that the typical range of calories in these snacks is between 300 kcal and 500 kcal, this error is extremely low. Consequently, the model demonstrates excellent predictive performance and generalizes well to unseen data, showing no signs of overfitting. We can conclude that this high accuracy is expected; indeed, the caloric content of food is directly derived from its macronutrient composition.